Chapter 5

Cross-validation

Amir Seraj[a], Mohammad Mohammadi-Khanaposhtani[b], Reza Daneshfar[c], Maryam Naseri[d], Mohammad Esmaeili[e], Alireza Baghban[f], Sajjad Habibzadeh[g], and Saeid Eslamian[h,i]

[a]*Department of Instrumentation and Industrial Automation, Ahwaz Faculty of Petroleum Engineering, Petroleum University of Technology, Ahwaz, Iran,* [b]*Fouman Faculty of Engineering, College of Engineering, University of Tehran, Tehran, Iran,* [c]*Department of Petroleum Engineering, Ahwaz Faculty of Petroleum Engineering, Petroleum University of Technology, Ahwaz, Iran,* [d]*Chemical Engineering Department, Babol Noshirvani University of Technology, Babol, Iran,* [e]*Department of Petroleum Engineering, Amirkabir University of Technology (Tehran Polytechnic), Tehran, Iran,* [f]*Chemical Engineering Department, Amirkabir University of Technology (Tehran Polytechnic), Mahshahr Campus, Mahshahr, Iran,* [g]*Surface Reaction and Advanced Energy Materials Laboratory, Chemical Engineering Department, Amirkabir University of Technology (Tehran Polytechnic), Tehran, Iran,* [h]*Department of Water Engineering, College of Agriculture, Isfahan University of Technology, Isfahan, Iran,* [i]*Center of Excellence in Risk Management and Natural Hazards, Isfahan University of Technology, Isfahan, Iran*

1. Introduction

1.1 Importance of validation

In our everyday life, we face questions like "How do you know that?," "Are you sure?," etc. But why these questions are asked so frequently? To see why let's consider the situations where such questions are made. When someone talks about an event, they give us a piece of "information" about a "result." Now if we would use this information in other situations, we need to make sure that it is "valid." It is always about losing or gaining some benefit by either retelling the story or acting based on that story. For example, we don't like to risk our reputation on unverified stories or lose our money in an investment based on unevaluated information. These examples were intended to demonstrate the importance of "validation." Every piece of information that tends to be useful in the future, needs to be validated first. In a scientific study, validation comes into play when some experimental data are acquired or when a model is proposed to generalize the applicability of this data. Especially, when we are looking for a predictive tool, validation becomes a crucial step in the development of model. For example, when a model is proposed for engineering or economic purposes, it must be validated before any design, investment, or forecasting upon that model. In science and engineering, when a model is derived by a mechanistic approach, a mathematical formula is obtained which relates the input to the specified output. In this respect, the validation could be made by comparison of the model results with accurate experimental data or at least by the model performance in the limiting situations, e.g., equilibrium or steady-state conditions. But there is an ever-increasing number of models based upon machine learning which try to offer predictive tools based on their training by the available experimental data. Here a valid model must act as well on the independent data sets, i.e., the data which is not incorporated in the derivation of the model.

1.2 Validation of the training process

In machine learning, the training phase usually takes the major portion of the available data (training data set) to find the model parameters that favorably minimize the error. Usually, the error rapidly decreases at the first iterations but as the training proceeds, the error would slowly decrease toward a local minimum (Richert, 2013). The training is stopped when the best generalization is achieved by the model which is the one that is not underfitting or overfitting the data. Since the training data set provides the expected output, the error could be minimized by a suitable method to achieve the best model parameters for the training data; however, if the model is so accurate for the training data set, it might act poor on the testing data set or any other future data (Fontaine, 2018). Thus, a stopping criterion is required to avoid overfitting in the model. Indeed, searching should be stopped when the criteria of error minimization and cross-validation are both met (Brownlee, 2018). For example, the validation of the model on the testing set could be performed during the training phase and when the

Handbook of HydroInformatics. https://doi.org/10.1016/B978-0-12-821285-1.00021-X
Copyright © 2023 Elsevier Inc. All rights reserved.

error of the predictions for the testing set reaches a local minimum the training would be stopped (Raschka, 2015). This type of validation which usually takes some 30% of the data, as the testing set is the simplest type of cross-validation but it still functions properly in any situation. However, for a more robust validation, more useful techniques might be incorporated which are classified and discussed in the subsequent sections (Lei, 2019; Berrar, 2019).

2. Cross-validation

The cross-validation is started by splitting the original data set into a training set(s) and test set(s). When all possible combinations of splitting the available data are considered, *exhaustive cross-validation* is going to be made (Brownlee, 2016). On the other hand, when all possible combinations are not considered a nonexhaustive, cross-validation is performed. The simple way of randomly splitting the data set into only one train set (usually containing 70% of the data) and one test set which contains the rest of the data, which is known as the hold-out method is an example of nonexhaustive cross-validation (Müller and Guido, 2016). Besides the hold-out method which is just considered a *simple validation* by some practitioners, the other cross-validation methods include multiple training/test sets and the final model is derived by averaging over the individual models obtained from different runs (Dangeti, 2017).

2.1 Exhaustive and nonexhaustive cross-validation

As suggested by its name, exhaustive cross-validation involves a great deal of computational effort as it considers all possible combinations of isolating a specified number of samples from the training phase to use them in the testing phase (Karim and Kaysar, 2016).

- **LpO-CV**

Generally, this type of cross-validation is addressed as leave-p out cross-validation (LpO-CV) since at each run, p samples are taken out of the original data set and the training is made by the remaining "n-p" samples where n represents the total number of samples in the original data set. Now if there are only 50 samples in the original data set, with $p = 15$, the total number of training-validation runs would be C(50,15) which is about 2.25×10^{12}. Selecting $p = 2$ provides the main advantages of this method for many problems (Brunton and Kutz, 2019).

- **LOO-CV**

A variant of LpO with $p = 1$ which is denoted as LOO-CV (leave-one-out cross-validation) is attracting more attention toward itself as it involves less computational effort while preserving some essential features of LpO-CV. That is so because this method provides "n-1" predictions for each sample in a data set of n samples and since the ratio is close to unity, the LOO-CV holds the variable section consistency in linear models (Viswanathan et al., 2016).

2.2 Repeated random subsampling cross-validation

This cross-validation scheme creates multiple random splits from the original data set. For each of these random splits, the training and testing are made to find the corresponding model and by averaging over these models, the final model is prepared. Since the selection of the training and testing sets is a random process, some of the samples may never be used in the validation stage while other samples might be used more than once. Also, the stratified version of this method is proposed for handling imbalanced data sets (Dua and Du, 2016).

2.3 Time-series cross-validation

For time-series or time-dependent data, the order of the data is of crucial importance that's why random splitting or k-fold partitioning may not provide satisfactory results. Instead, the data split is made based on time and the training is performed using the prior subsets with the next subset as a validation set. This method is referred to as rolling cross-validation and also the walk-forward or forward chaining method. Note that this method prevents the leakage of data from the future to the training set which makes the rolling CV stand out (De Prado, 2018; Jansen, 2018; Hadizadeh and Eslamian, 2017).

2.4 k-fold cross-validation

The LOO-CV requires n runs which is quite large for many problems thus for saving computation effort, the k-fold technique was developed. This method partitions the data set in k equal size subsets or folds. At each pass, "k-1" folds are used for the training and the remaining fold takes the rule of the testing set (Géron, 2019). The process is repeated k times until

each of the *k* folds has been once used as the test set. Finally, the model is derived by averaging the results of different runs. Although LOO-CV has been considered as a variant of LpO-CV, one might assume that LOO is a form of k-fold cross-validation with k = n (Kumar, 2019; Kane, 2017).

2.5 Stratified k-fold cross-validation

For imbalanced data sets, containing two or more different classes with a different numbers of data points the naïve k-fold CV does not work properly. The solution is sought by partitioning the data set in a way that the mean response value in all folds to be nearly equal. This method is known as stratified k-fold cross-validation. Note that this method is not suitable for time series data sets (Swamynathan, 2019; El Naqa et al., 2015).

2.6 Nested

Finally, the k-fold cross-validation could be run in a nested scheme if both hyperparameter selection and error minimization are planned to be made simultaneously. For example, in the k*l-fold cross-validation method, an outer loop and an inner loop are designed to cover both tasks. Here the outer loop makes the usual k-fold cross-validation, while the inner loops are responsible for fitting the model parameters. Note that the "l" in the name of the method refers to the number of subdivisions of outer training sets (Raschka and Mirjalili, 2017; Deisenroth et al., 2020).

3. Computational procedures

To comprehend completely and dive deep into cross-validation, the problem of predicting Boston house pricing is brought here. As mentioned in previous sections, the difference between a regression and classification case is the output of these types of problems. In a regression case, the output continues, the output is a continuous numeric variable, some instances for this case study are predicting the temperature tomorrow or another instance is predicting the price of a stock, given its previous prices. Another example for this case is the prediction of time which a software will terminate by giving its specifications. In this chapter, we want to check out the case of predicting the price of homes in Boston suburbs. In the 1970s, as it can be understood, the home price is a continuous variable. Some features in this case study are crime rate, the average number of rooms in the house and accessibility to different highways from house and so forth. In this example, the concept of k-fold cross-validation will be investigated practically. Data for the Boston problem exist in the Keras library. For solving this case, Spyder IDE of Python was selected. Python has several IDEs such as Spyder, Atom, PyCharm, and so on. For machine learning and deep learning tasks, Spyder and Jupyter IDEs are utilized more and have special applications for machine learning tasks in comparison with other IDEs. Spyder IDE is selected to solve the Boston problem. Let's dive into the solution. It is noted that all codes are written in Spyder IDE (Zaman et al., 2019).

```
(1) from Keras.datasets import boston_housing
```

The first Boston dataset is imported. As mentioned before, we used Keras datasets. Boston_housing is a dataset in Keras library.

```
(2) (train_data, train_targets ), (test_data, test_targets) = boston_housing.load_data();
```

For loading data and splitting data into train and test data, a function is applied known as the load_data() function. The definition of load_data() function is as follow:

```
load_data(path='boston_housing.npz', test_split=0.2, seed=113)
```

The first argument of this function is the path. The path is the directory where to cache the dataset locally. The first time a user wants to use a dataset in Keras, the Spyder download it automatically by load_data() function and saves this dataset in the path (relative to ~/.keras/datasets) as locally (Sarkar et al., 2018). Another argument is test_split which means a fraction of data to reserve as test data and seed means random seed for shuffling the data before computing the test split. The output of this function is returned as a tuple of Numpy arrays: (x_train, y_train), (x_test, y_test). In line 2, train and test targets are the labels of train and test data. Train or test data gives data or features and train and test targets give labels of these data.

```
(3) train_data.shape => Out[3]: (404, 13)
```

After executing the code in line 3, the output is: Out[4]: (404, 13). It means, this dataset has 404 train data and there are 13 features which as mentioned before some features are such crime rates, accessibility to highways, the number of rooms which every house has, etc.

```
(4) test_data.shape => Out[4]: (102, 13)
```

This means, 102 data is randomly selected for test data while like train data, it has 13 features. As it is obvious, the deep learning model wants to predict the price of houses, therefore, the output of the model is the price of houses in Boston.

```
(5) train_targets => Out[5]: array([15.2, 42.3, 50. , 21.1, 17.7, .........,19.4, 19.4, 29.1])
```

By executing line 5, the output is like Out[5], which means the price of the first house or first district in Boston city in the dataset is 15.2 thousand dollars and the second sector is 42.3 thousand dollars, etc. These prices are the median values in each city district in thousands of dollars. One important point is about features and that means the range of features is different, in another word, each feature has a different scale or different range, some are between 1 and 12, some between 0 and 1, others between 0 and 100, and so on. In general, there are 13 features which each feature has a different range. This point is too significant which before starting operation for training data, the range of all data should be the same, and in many cases, the range of data should be between 0 and 1. This operation is called data normalization. If don't do the preprocessing operation, the learning rate becomes too slow and hard to train the model. One important factor in data preprocessing is normalization. This means all data features should be in the same range. One specific type of normalization is called *Z*-score normalization. *Z*-score normalization means each feature centered zero and its standard deviation should be unit (Han et al., 2011). This causes all features to have the same distribution. If the mean of all data becomes one specific number and the standard deviation becomes that specific number, the result becomes the same distribution. The mean of train data is computed in line 6 by the mean() function and store into a mean variable.

```
(6) mean = train_data.mean(axis=0)
```

Axis = 0 in mean function means the averaging operation is executed on each column. As mentioned before each column is an indicator of a specific feature. The output of code in line 6 is a vector of 13.

```
   Out[6]: array([3.74511057e+00, 1.14801980e+01, 1.11044307e+01, 6.18811881e-02,
     5.57355941e-01, 6.26708168e+00, 6.90106436e+01, 3.74027079e+00,
     9.44059406e+00, 4.05898515e+02, 1.84759901e+01, 3.54783168e+02,
     1.27408168e+01]).

(7) train_data - = mean;
```

In code of line 7, the mean value is reduced from all of the train data. This causes the mean of train data to become zero. Lines 6 and 7 cause to mean of train data becomes zero. Now turn to standard deviation (std), standard deviation should be unit.

```
(8) std = train_data.std(axis=0);
(9) train_data /=std;
```

Lines 8 and 9 make the standard deviation of train data become a unit. If take a standard deviation from train data, it returns a unit. Out[10] proves this issue. As it can be seen it returns a unit array. When a number is divided into standard deviations, the result turns to one.

```
(10) In[10] : train_data.std(axis=0)

   Out[10]: array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

As a result of *Z*-score normalization the mean is zero and the standard deviation (std) should be united.

```
(11) test_data -=mean;
(12) test_data /=std;
```

If consider with details to the python codes, lines 11 and 12, mean and std are computed from train data. It is noted that for test data, the mean and std of train data are being used. It should be considered that when users wants to normalize test data, this normalizing process should be done by training data quantities (mean and std of train data). But a question here arises as that why for normalizing test data, data information (mean and std) of train data is used for normalization and it doesn't use from test data information (mean and std) for normalizing test data. The reason is that when preparing the model and a new case (a new home) comes into the model to predict its price, the first problem created here is that, it is impossible to compute

the standard deviation and mean for one test data. The second problem is that another name of test data is unseen data. It means that when the model is training, the user shouldn't give any information about test data because in the real world you don't have any information about the new case (test data) which comes in the model. Therefore, it is not allowed to compute the std and mean from test data because it is a type of cheating. It seems to give some information about test data to your model. After loading (preparing) and normalizing or preprocessing data, we should construct network architecture.

Network architecture

First, it should be import relative libraries from Keras library such as models and layers to construct the model. Lines 13 and 14 import relative libraries (models and layers) from Keras.

```
(13) from keras import models
(14) from keras import layers
```

In line 15, define a function known as build_model() with keyword def, we going to construct the model into the build_model() function. The intention for defining a function is that the constructed model is going to apply in different places in the program. Line 16 makes a sequential model. The sequential model is favorable for a plain stack of the layer in which each layer has one input tensor and one output tensor (Krohn et al., 2019). This module is presented in the engine of Keras. Add function is present in the engine of Keras in the Sequential section.

```
(15) def build_model():
(16) model = models.Sequential()
```

From layers library in add function created a dense layer with 64 neurons and with an activation function relu. This point is important to mention that train data has two dimensions (a two-dimension tensor). The first dimension is the number of data (number of rows) and the second dimension is the number of features (number of columns). Therefore train_data.shape[0] is the number of rows (data) and train_data.shape[1] is number columns (features).

```
(17) model.add(layers.Dense(64,activation='relu',input_shape=(train_data.shape[1],)))
```

A definition of activation functions

Activations can either be used through an "Activation" layer, or through the "activation" argument that is supported by all forward layers. There are many activation functions such as elu, selu, softmax, softplus, tanh, sigmoid, hard_sigmoid, softsign, exponential, linear and relu. Fig. 1 indicates the graph of relu activation function (Millstein, 2020).

In line 18, another dense layer is added with 64 neurons.

```
(18) model.add(layers.Dense(64,activation='relu'))
```

As additional explanation about optimizers such as relu and function of each layer consider Fig. 2.

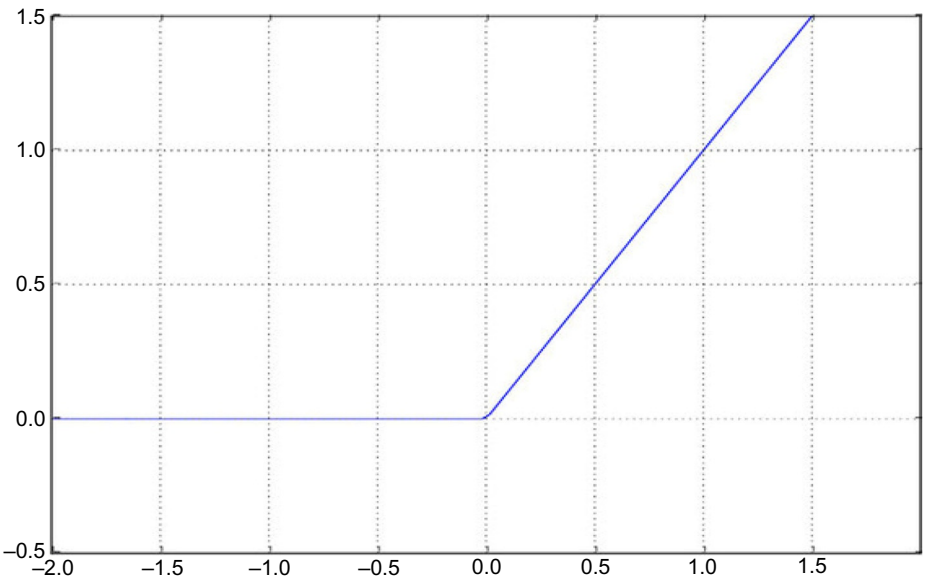


FIG. 1 Relu: the rectified linear unit function.

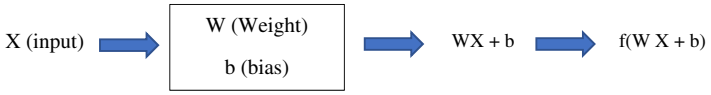


FIG. 2 The layer explanations.

X (data) is the input of layer; each layer has two characteristics such as weight (w) and bias (b). The output of layer is WX+b which is a linear equation. WX+b as an input is given to a nonlinear function such as relu, elu, softmax, and so forth to become nonlinear. This nonlinear function can be modeled every nonlinear system and is the base of Artificial Neural Networks (ANN). In linear layers, there is no f as f(W X+b) and is just a linear function W X+b.

In line 19, another dense layer is added with one neuron (Moolayil et al., 2019). Why there is one neuron in the output layer? As mentioned before, the goal of the Boston problem is predicting house prices in Boston district. As a result, the output of this problem is one variable (house price in Boston); therefore, there is one neuron in the output layer. The important point here, is that, there is no activation function in the output layer. When don't put any activation function in the output layer, the activation function is assumed linear. But the question here arises why don't use any activation function in the output layer. The answer is that approximately all activation functions limit the output. For instance, the softmax activation function, reduce the range of output to [0 1]. In this case, we intend to predict the price of the house and this price can be in any range of price (can be any number); therefore, the last layer has no activation function. It will be a linear layer free to learn to predict values in any range. Generally, for regression problems, the activation function for the output layer is linear. Line 9 indicates this issue.

```
(19) model.add(layers.Dense(1))
```

The next step is assigning optimizer, loss function, and metrics.

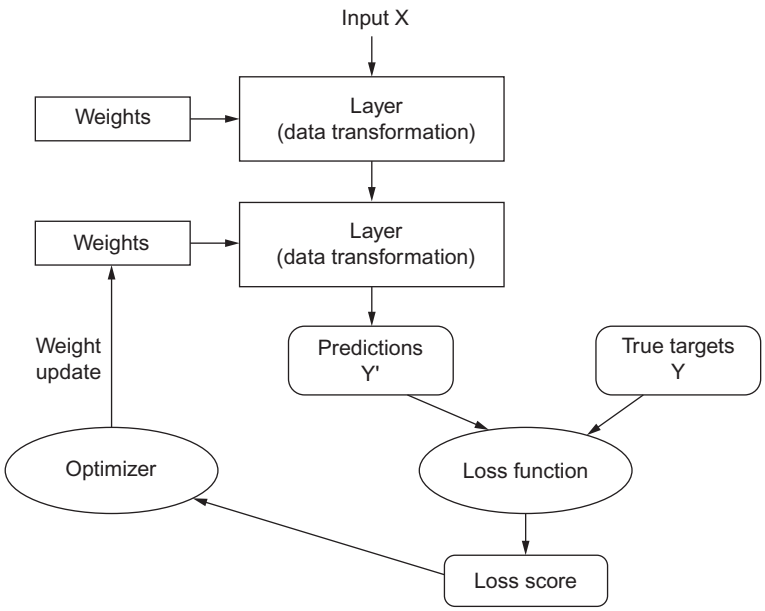
```
(20) model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
```

In this step, and as a explain for codes in line 20, the model should be compile via compile method. Each compile method has several parameters such as optimizer, loss and metrics. The definition of compile method is bright in bellow:

```
compile(self, optimizer, loss=None, metrics=None, loss_weights=None,sample_weight_mode=None,
weighted_metrics=None, target_tensors=None, **kwargs)
```

First, it is necessary to define optimizers, the duty of optimizers is to change the value of the weight a bit based on loss values to decrease the loss value. Fig. 3 shows this mechanism clearly. This figure indicates the relationship between the layers network, loss function, and optimizer. The adjustment between these two numbers is the job of the optimizer, which this operation is implemented by the Backpropagation algorithm: the principle and central algorithm in deep learning (DL) (Goodfellow et al., 2016).

FIG. 3 The loss value is used as a feedback signal to regulate the weights of layers in an appropriate direction. This regulation is executed via optimizers.



In this program, used from rmsprop, it is suggested to leave the parameters of this optimizer at their default values (except the learning rate, which can be freely tuned). This optimizer is a good choice for recurrent NNs. Another parameter of compile function is assigning the loss function. The loss function is an object that the model try to minimize. As mentioned before the loss function is one of the two parameters required to compile the model. Different types of loss function based on types of the problem; the used loss function is different. For example, the "mse" loss function is usually applied for regression problems. For classification problems, if there is a binary problem "binary_crossentropy" is applied and if there are multi classes in output, categorical_crossentropyp is applied. In line 20, the loss function is assigned mse because as mentioned before this case is a regression problem (Shanmugamani, 2018). What is the definition of mse? The abbreviation of mse is mean square error. In statistics, the mean square error (MSE) or mean square deviation (MSD) of an estimator (a procedure for estimating an unobserved quantity) measures the average of the squares of the errors, it means, the average squared difference between the estimated values and the actual value. In Eq. (1), the MSE equation is bright:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \widehat{Y}_i\right)^2 \tag{1}$$

n = number of data points
Y_i = observed value
$\widehat{Y}_i$ = predicted value

Another parameter of compile function is metrics; a metric is a function that is used to determine the functionality of the model. The performance of metric function is similar to the loss function, except that the results from estimating a metric are not used when training the model. There are multiple different metrics such as binary_accuracy, categorical_accuracy, and so on. The metric applied in this case is MAE, what is MAE? Mae stands for mean absolute error. Mae is approximately similar to mse with a little difference. Eq. (2) indicates the MAE.

$$\mathrm{MAE} = \frac{\sum_{i=1}^{n}\left|y_{i-x_i}\right|}{n} = \frac{\sum_{i=1}^{n}|e_i|}{n} \tag{2}$$

In classification problems, accuracy is applied to record output while in regression problems, accuracy record is meaningless. To record the output and performance of a regression program MAE is recorded more. For instance, an MAE of 0.5 on this program would mean your predictions are off by 500 dollars on average. As mentioned later in k-fold cross-validation, every fold should be generated by the model. In order to avoid repetition in the program, all codes associated with the model have been put into one function. In code 21, this function is defined completely.

```
(21) def build_model():
     model = models.Sequential()
     model.add(layers.Dense(64,activation='relu',input_shape=(train_data.shape[1],)))
     model.add(layers.Dense(64,activation='relu'))
     model.add(layers.Dense(1))
     model.compile(optimizer='rmsprop',loss='mse',metrics=['mae'])
     return model
```

- K-fold cross-validation

There are several ways for making validation data, one manner for validation data is splitting train data into two sections (train and validation data) (Dubitzky et al., 2007; Gollapudi, 2016). This method is a so-called hold-out method. When the number of data is small like the Boston problem (in this problem the total of data is 400), this method (hold-out method) is not efficient for this problem. Splitting this little data into two sections and the separated train data is not reliable as the performance of the model. This method is not efficient for little data. A way for solving such problems is using K-fold cross-validation (Provost and Fawcett, 2013). The procedure of K-fold is that data is divided into k folds equally. For instance in Fig. 4, it can be seen a threefold cross-validation. After dividing data into k folds, the model is run k times, for example, three times. In run 1 or fold 1, part 1 becomes validation and the two successive parts become training and run 2 and run 3 are executed respectively by moving the validation section into the data set. In each fold, the score of the model (validation score) is assessed and reported and finally the average of these scores is measured. The average of these scores is assumed as model accuracy. As mentioned before, the idea of K-fold cross-validation is appropriate for little datasets such as the Boston

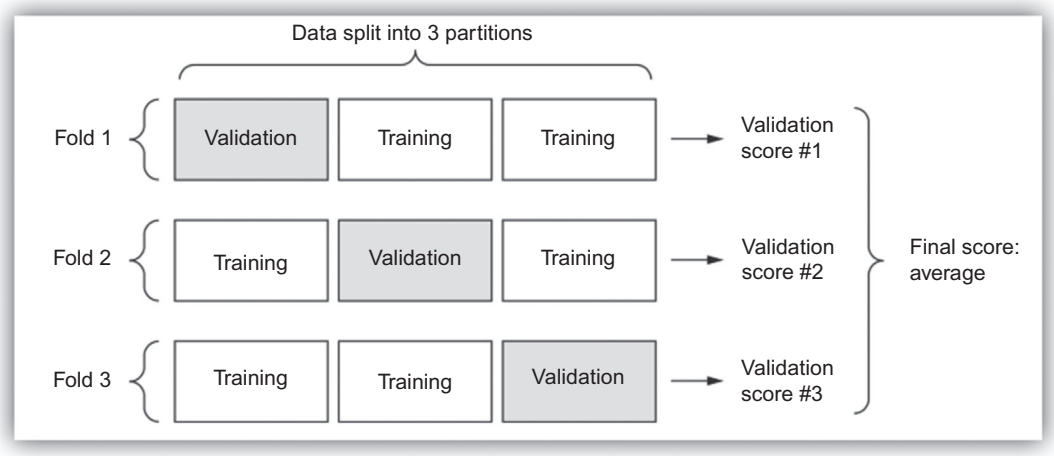


FIG. 4 The idea of K-fold cross-validation.

dataset. The computational process becomes difficult in K-fold cross-validation in terms of repeating model training because it should be trained k different models while in previous cases, it trained one model and then evaluated it with a validation set but the K-fold method is more reliable in comparison with other methods particularly for little datasets. Fig. 4 indicates the idea of K-fold cross-validation in detail.

The following explained the codes about k-fold cross-validation in detail and clearly. In this example, assumed there are four folds cross-validation.

```
(22) import numpy as np
(23) k=4 (number of folds)
(24) num_val_samples = len(train_data) // 4 => at this line the number of validation samples are calculated. Please noted that the sign
     // means correct division. For instance 5/4 = 1.25 but 5//4 = 1, it turns out an integer number.
(25) num_epochs = 500 => number of epochs.
(26) all_scores = [ ] => this array stores the score of each fold and consequently the length of this array will be 4.
```

Now we should prepare data: in lines 27 and 28 validation data and targets (labels) are prepared separately. For example

val_data when i=0 is val_data = train_data[0:101] and the validation targets are: val_targets = train_targets[0:101]

```
(27) val_data = train_data[i * num_val_samples : (i+1) * num_val_samples]
(28) val_targets = train_targets[i * num_val_samples : (i+1) * num_val_samples]
```

In lines 29 and 30 prepared partial train data and partial train targets

```
(29) partial_train_data = np.concatenate(
          [train_data[:i*num_val_samples],
          train_data[(i+1)*num_val_samples:]],
          axis=0)


(30) partial_train_targets = np.concatenate(
          [train_data[:i*num_val_samples],
          train_data[(i+1)*num_val_samples:]],
          axis=0)
```

to explain code lines 29 and 30 assume i=1 => to simplify our task lets abbreviate num_val_samples ~ nvs: therefore validation data is => `val_data = train_data[nvs : 2*nvs]` while `partial_train_data = train_data[:nvs]` and `train_data`[2*nvs:].concatenate means attach these two ranges of data together. Now to make a new model, call build_model() function:

```
(31) model = build_model()
```

The model should now be trained using the partial_train_data by fit function. As mentioned before training process is executed by fit function. This function have several parameters:

```
fit(self, x=None, y=None, batch_size=None, epochs=1, verbose=1,callbacks=None, validation_split=0.,
validation_data=None, shuffle=True,class_weight=None, sample_weight=None, initial_epoch=0,
steps_per_epoch=None,validation_steps=None, validation_freq=1, max_queue_size=10,
workers=1,use_multiprocessing=False, **kwargs)
```

```
(32) model.fit(partial_train_data,
          partial_train_targets,
          epochs=num_epochs,
          batch_size=1,
          verbose=0)
```

As it is obvious the number of epochs are 500 and the important point is that the verbose parameter is zero, but the question here is what is the use of verbose while training the model? By setting verbose 0, 1 or 2, you mention that how do you want to see the training progress for each epoch.

Verbose = 0 will show you nothing(silent)
Verbose = 1 will show you an animated progress bar like this:

[==============================]

Verbose = 2, will just mention the number of epoch like this: Epoch 1/10

Therefore, the code line 32 executed the training operation on the train data. Now the trained model should be evaluated on the validation data and then the mae (mean absolute error) which is measured from this step appended to all_scores list (array).

```
(33) val_mse, val_mae = model.evaluate(val_data,val_targets,verbose=0)
(34) all_scores.append(val_mae)
```

Because k = 4, the loop is repeated four times and as a result four mae will be obtained for validation data and the final mae will be the average of all maes in all_scores list. It is noteworthy to mention that the subject of k-fold cross-validation is too significant in machine learning and data mining. It will be useful to mention that all codes about k-fold cross-validation are ready in scikit to learn and machine learning libraries and there is not necessary to write codes. Now run code and print all_scores:

```
(35) print(all_scores) => [2.018010139465332, 2.5180163383483887, 2.4935202598571777,
          2.712078809738159] => while the number of epochs are 500
```

As it is obvious, there are four folds and for each fold, there is one mae. As mentioned before, we defined an array known as all_scores and save all folds maes in this array. If averaged from these maes.

```
(36) np.mean(all_scores) => 2.4354063868522644
```

This means the average mean absolute error among all folds is approximately 2.43. the idea of k-fold is that instead of making train and validation data one time, the train and validation data are made several times and then take an average from those maes. In this state, the output of this method is more reliable. If attend to codes, the final result of the program are one mae. If we want to record the status of program in each fold and all steps (epochs) and save them (for instance if the

program has 500 epochs this manipulated code, saves 500 maes), we should manipulate the codes a bit. The output of this code is a matrix with 500 rows and four columns.

```
(37) history = model.fit(partial_train_data,partial_train_targets,
            validation_data=(val_data, val_targets),
            epochs=num_epochs,
            batch_size=1,
            verbose=0)
     mae_history = history.history['val_mean_absolute_error']
     all_mae_histores.append(mae_history)
```

mae_history are saved in an array known as all_mae_history but first we should define it. =>

```
all_mae_history = [ ]
```

The output of this code (line 37) is an array that has 4 elements and every element in this array is a list (array) again. This gives 500 mean absolute error (mae) in each epoch. The array is like this:

$[[MAE]_{500*1}, [MAE]_{500*1}, [MAE]_{500*1}, [MAE]_{500*1}]$

Each list (array) in this list is for one specific fold. Let's increase epoch number to 500. To consider precisely, should be plotted the output of the program (validation MAE). But what this figure means? For more explanations, consider the list below:

$$\begin{aligned}
&\text{Fold1} \quad [[MAE_1, MAE_2, MAE_3, \ldots\ldots\ldots\ldots\ldots\ldots, MAE_{500}],\\
&\text{Fold2} \quad [MAE_1, MAE_2, MAE_3, \ldots\ldots\ldots\ldots\ldots\ldots, MAE_{500}],\\
&\text{Fold3} \quad [MAE_1, MAE_2, MAE_3, \ldots\ldots\ldots\ldots\ldots\ldots, MAE_{500}],\\
&\text{Fold4} \quad [MAE_1, MAE_2, MAE_3, \ldots\ldots\ldots\ldots\ldots\ldots, MAE_{500}]]
\end{aligned}$$

When calculating the model performance after epoch 1, you should average all MAE_1 values. All codes about this operation and plotting the results are reported as bellow:

```
(38) average_mae_history=[np.mean([x[i] for x in all_mae_scores]) for i in range(num_epochs)]
(39) plt.plot(range(1,len(average_mae_history)+1),average_mae_history)
     plt.xlabel('Epochs')
     plt.ylabel('Validation MAE')
     plt.show()
```

If consider closely to Fig. 5, MAE is decreased rapidly in earlier epochs and little by little begins to increase and start to overfits. If attended to this figure about epoch 80, model started to overfit.

Two changes enabling us to recognize the variations in the plot are applied. First removing the first epochs because they don't have any benefits and another change, removing sharp changes in plot, this task is so-called plot smoothing. One way for plot smoothing and removing noise from the plot is averaging. For removing the sharpness in the plot and smoothing it, the code below is written.

```
(40) def smooth_curve(points,factor=0.9):
          smoothed_points=[]
        for point in points:
          if smoothed_points:
          previous = smoothed_points[-1]
          smoothed_points.append(previous*factor+point*(1-factor))
       else:
          smoothed_points.append(point)
      return smoothed_points


(41) smoothed_mae_history=smooth_curve(average_mae_history[10:])
(42) plt.plot(range(1,len(smoothed_mae_history)+1),smoothed_mae_history)
    plt.xlabel('Epochs')
    plt.ylabel('Validation MAE')
    plt.show()
```

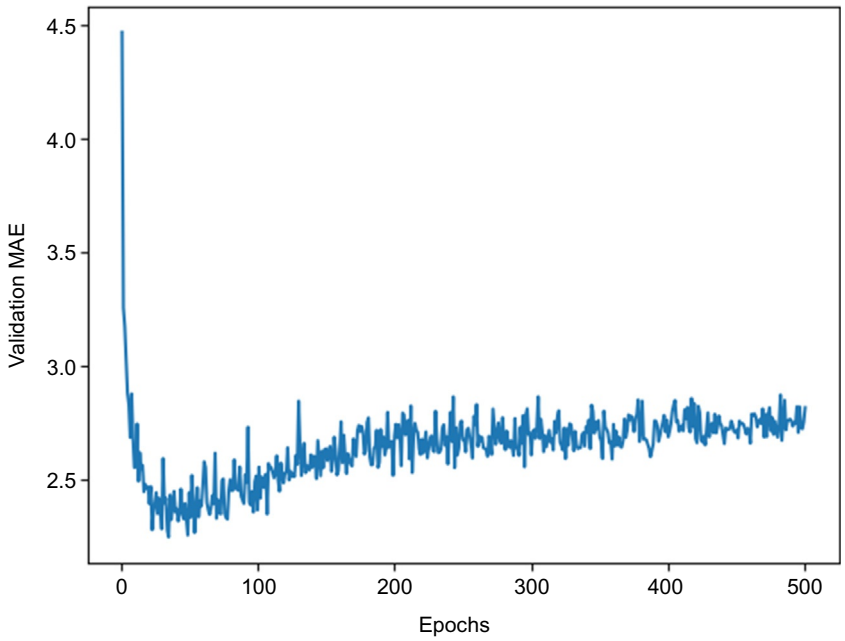


FIG. 5 Output of network (Validation MAE per Epochs).

First, in line 40, the smooth_curve is defined; the duty of this function is removing sharpness from validation MAE plot. Then in line 41, all data from epoch 10 passed to this function and in line 42, the diagram is plotted via matplotlib library (this library is in core of python for plotting different diagrams). The output of this code is much clearer and more explicit than previous one. As it can be seen from this plot after epoch 40, the network is starting to overfit; therefore, the epoch 40 is the best situation for stop training. Fig. 6 indicates the output of network while epochs 1–10 are omitted and plot are smoothed by applying smooth function. The average type which applied in this example is exponential moving average (Khosrow-Pour, 2012).

After determining the hyper parameter (epochs number) in previous step, now should be retrain the network on all train data with 40 epochs. This task executed with fit method. Then the retrained network is evaluated on test data and finally reported the test_mae_score.

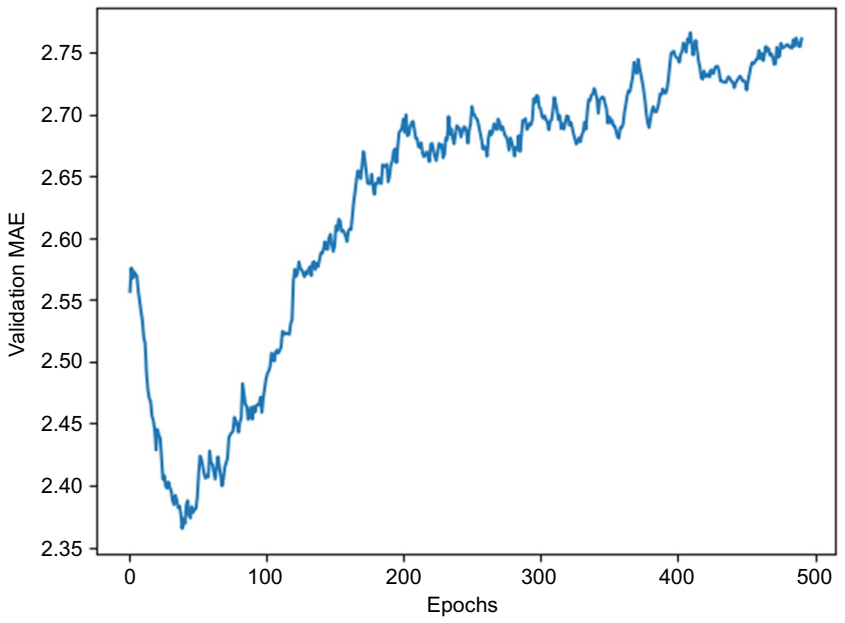


FIG. 6 Validation MAE after applying smooth_curve function on data.

```
(43) model=build_model()
(44) model.fit(train_data,train_targets,epochs=40,batch_size=16,verbose=0)
(45) test_mse_score,test_mae_score=model.evaluate(test_data,test_targets)
```

```
>>>> test_mae_score = 2.845837354660034
```

This means the error of network is about 2.8, in another word the error of this network is 2.8 thousand dollars. For instance, if the price of a house is $15,000, this network predicted the price between about $12,500 and $17,500.

Now we are going to go through a project called nutrient removal efficiency data and we use a data set containing 7876 data to analyze the total nitrogen (TN) removal efficiency of an anaerobic anoxic-oxic membrane bioreactor system. About 5000 data were separated for training the network and the remaining were allocated for test data. In this example, we used K-fold cross-validation method for training network. The output values are predicted by 9 input data presented in Table 1. This dataset was taken from the data reported from a paper published by Yaqub et al. in 2020 (Brownlee, 2018).

At first, we consider K = 2, and then the K-fold cross-validation method was executed on the training data. According to Fig. 7, visually we can understand from epochs 70, the overfit related to this model begins.

To remove the figure oscillations, the smooth function was used to view the overfit point more clearly. It is concluded that the overfit value begins from epochs 75 which is shown in Fig. 8.

After magnifying the curve of Fig. 8 around the overfit point, the exact value of the overfit point is epochs 75 which can be seen in Fig. 9.

After completing the run of cross-validation code, the test MAE and MSE values are obtained:

```
test_mae_score: 3.131399154663086
test_mse_score: 21.55681976617551
```

In the second run, we consider K = 5. After training the network via the K-fold cross-validation code, Figs. 10–12 were obtained. By considering these figures, after epochs 47, the overfit starts.

After setting the epochs on 47 and compiling the network again on the test data, the MAE and MSE values are obtained:

```
test_mae_score: 2.9801080226898193
test_mse_score: 19.03835214353075
```

Finally, after the complete run of the cross-validation code with k = 7, it was concluded that the start of overfitting is from epochs 40 and the following diagrams are reported for the output (Figs. 13–15):

TABLE 1 Attribute information of nutrient removal efficiency project.

Code	Input or output	Description
TOC	Input	Total organic contents
TN	Input	Total nitrogen
TP	Input	Total phosphorous
COD	Input	Chemical oxygen demand
NH4-N	Input	Ammonium
SS	Input	Suspended solids
DO	Input	Dissolved oxygen
ORP	Input	Oxidation-reduction potential
MLSS	Input	Mixed liquor suspended solids
RE of NH4-N	output	Removal efficiency of NH4-N
RE of TN	output	Removal efficiency of TN
RE of TN	output	The removal efficiency of TP

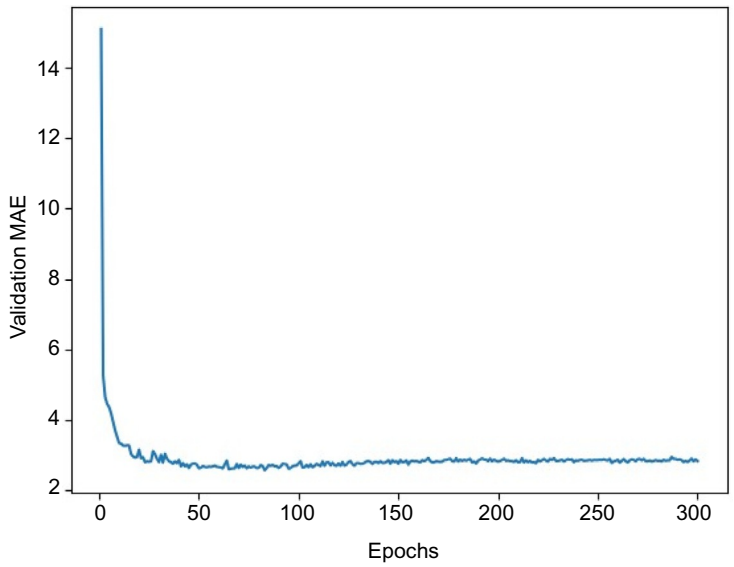


FIG. 7 The output of network (Validation MAE per Epochs).

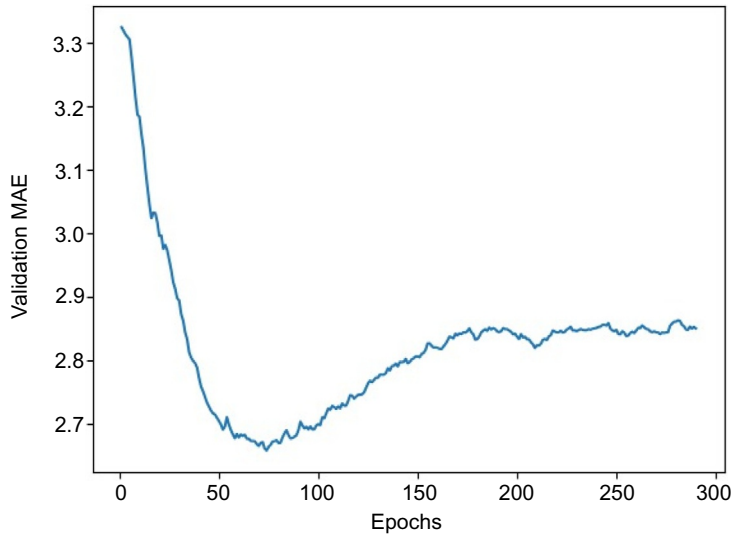


FIG. 8 Validation MAE after applying smooth_curve function on data.

After completing the run of network on test data for epochs 40, the MAE and MSE values are reported as below.

```
test_mae_score: 2.796928882598877
test_mse_score: 15.402697095683976
```

By comparing the values obtained from the data related to this project, it can be inferred that with increasing k, the mae (and also mse) values decrease. Of course, it should not be forgotten that with increasing k, the computational time increases sharply and a logical balance must be struck between these two inverse factors.

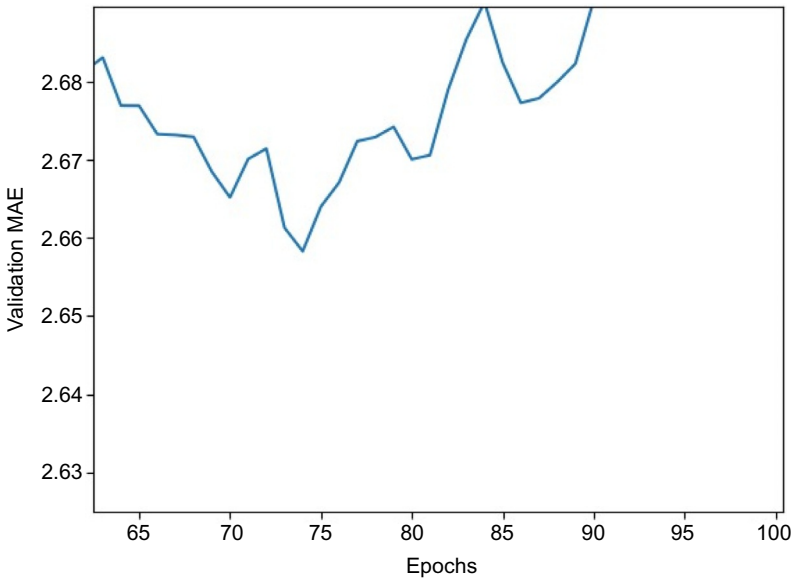


FIG. 9 Validation MAE after applying smooth_curve function on data and magnifying around overfit point.

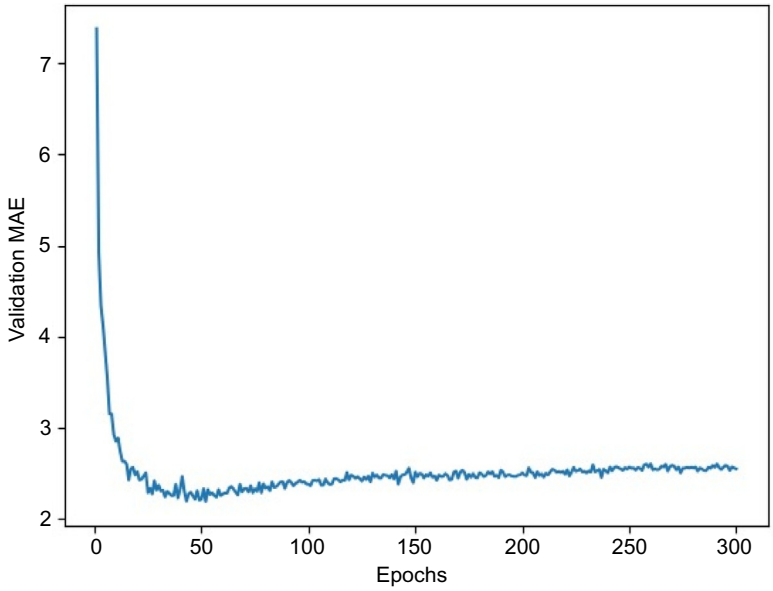


FIG. 10 The output of network (Validation MAE per Epochs).

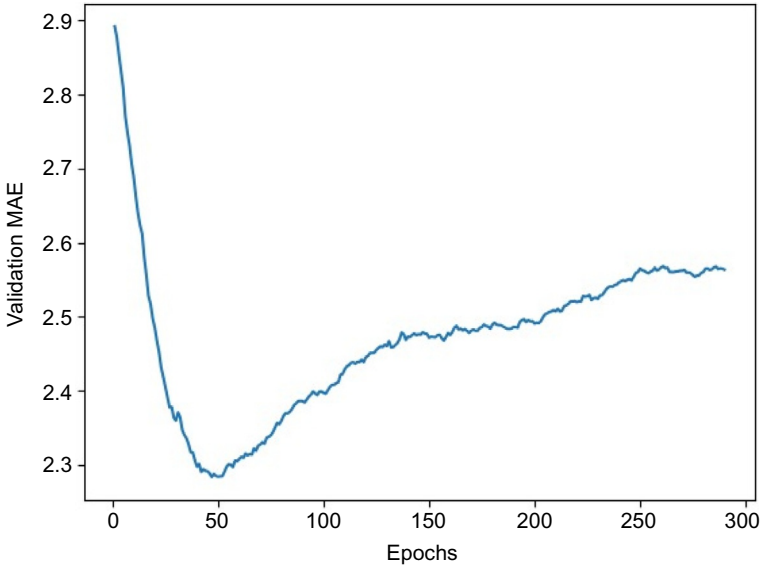


FIG. 11 Validation MAE after applying smooth_curve function on data.

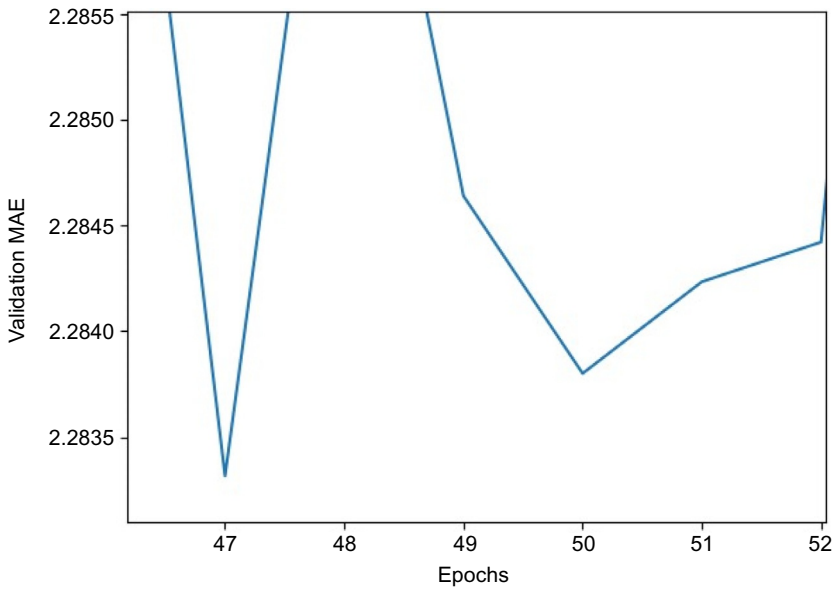


FIG. 12 Validation MAE after applying smooth_curve function on data and magnifying around overfit point.

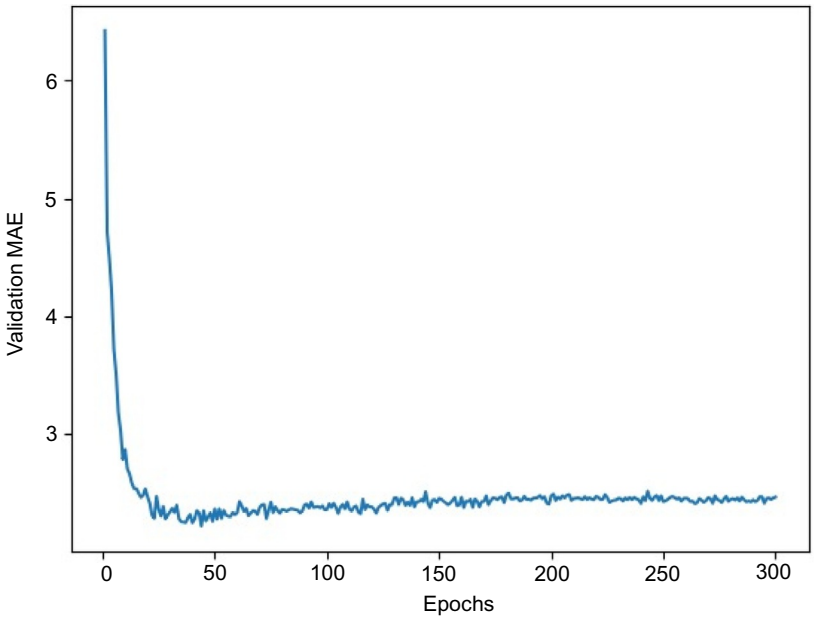


FIG. 13 The output of network (Validation MAE per Epochs).

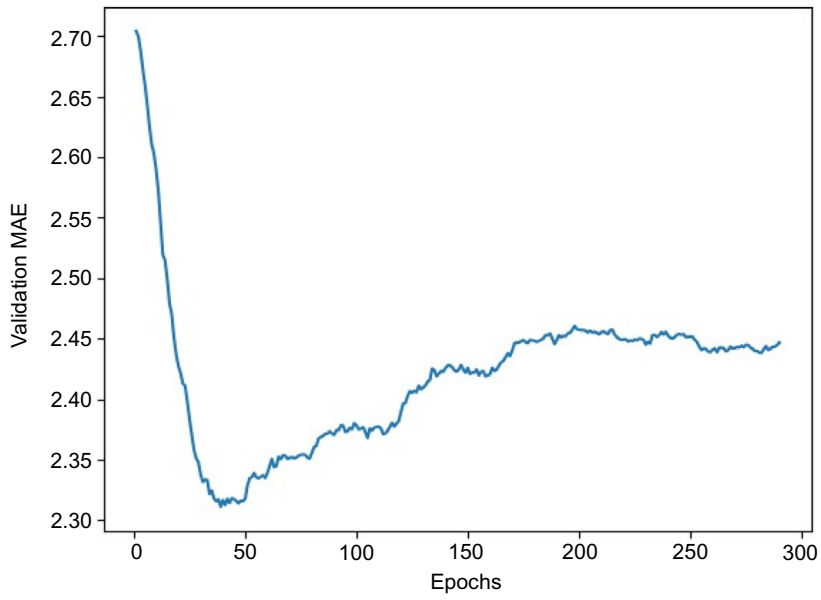


FIG. 14 Validation MAE after applying smooth_curve function on data.

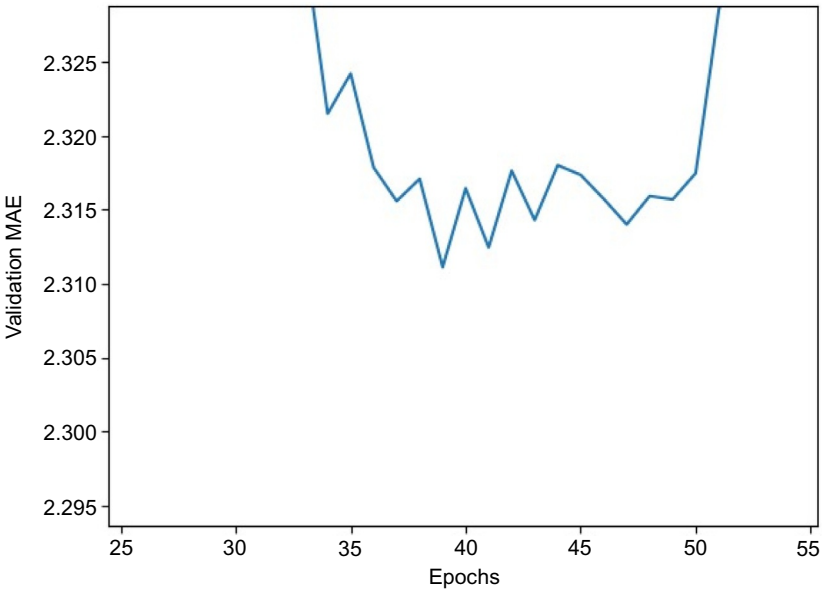


FIG. 15 Validation MAE after applying smooth_curve function on data and magnifying around overfit point.

4. Conclusions

In this chapter, the K-fold cross-validation is explained in detail and a famous and familiar example about this issue (predicting Boston house pricing) is represented. As mentioned before, there are two approaches for cross-validation types of problems: the first method uses ready-made machine learning library in python known as scikit-learn and the second one codes cross-validation in python from scratch. For more understanding about cross-validation concept, we have chosen the secondary method named deep learning technique. The achieved result in Boston indicated that the value of the mean absolute error is about 2.84 which means the predicted price has difference about $2800 in comparison with its actual value. Also, by analyzing the data of the nutrient removal efficiency project, it was concluded that with increasing k, the obtained MAE values decrease and the time for calculations increases sharply.

References

Berrar, D., 2019. Cross-validation. In: Encyclopedia of Bioinformatics and Computational Biology. vol. 1. Elsevier, pp. 542–545.

Brownlee, J., 2016. Machine Learning Mastery With Python. Machine Learning Mastery Pty Ltd, pp. 100–120.

Brownlee, J., 2018. Statistical Methods for Machine Learning: Discover How to Transform Data Into Knowledge With Python. Machine Learning Mastery.

Brunton, S.L., Kutz, J.N., 2019. Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control. Cambridge University Press.

Dangeti, P., 2017. Statistics for Machine Learning. Packt Publishing Ltd.

De Prado, M.L., 2018. Advances in Financial Machine Learning. John Wiley & Sons.

Deisenroth, M.P., Faisal, A.A., Ong, C.S., 2020. Mathematics for Machine Learning. Cambridge University Press.

Dua, S., Du, X., 2016. Data Mining and Machine Learning in Cybersecurity. CRC Press.

Dubitzky, W., Granzow, M., Berrar, D.P., 2007. Fundamentals of Data Mining in Genomics and Proteomics. Springer Science & Business Media.

El Naqa, I., Li, R., Murphy, M.J., 2015. Machine Learning in Radiation Oncology: Theory and Applications. Springer.

Fontaine, A., 2018. Mastering Predictive Analytics With Scikit-Learn and TensorFlow: Implement Machine Learning Techniques to Build Advanced Predictive Models Using Python. Packt Publishing.

Géron, A., 2019. Hands-on Machine Learning With Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. O'Reilly Media.

Gollapudi, S., 2016. Practical Machine Learning. Packt Publishing Ltd.

Goodfellow, I., et al., 2016. Deep Learning. vol. 1 MIT Press, Cambridge.

Hadizadeh, R., Eslamian, S., 2017. Modeling hydrological process by ARIMA–GARCH time series. In: Eslamian, S., Eslamian, F. (Eds.), Handbook of Drought and Water Scarcity, Vol. 1: Principles of Drought and Water Scarcity. Taylor and Francis, CRC Press, USA, pp. 571–590 (Chapter 30).

Han, J., Pei, J., Kamber, M., 2011. Data Mining: Concepts and Techniques. Elsevier.

Jansen, S., 2018. Hands-On Machine Learning for Algorithmic Trading: Design and Implement Investment Strategies Based on Smart Algorithms That Learn From Data Using Python. Packt Publishing Limited.

Kane, F., 2017. Hands-on Data Science and Python Machine Learning. Packt Publishing Ltd.

Karim, M.R., Kaysar, M.M., 2016. Large Scale Machine Learning with Spark. Packt Publishing Ltd.

Khosrow-Pour, M., 2012. Machine Learning: Concepts, Methodologies, Tools and Applications. Information Science Reference, Hershey, PA, USA.

Krohn, J., Beyleveld, G., Bassens, A., 2019. Deep Learning Illustrated: A Visual, Interactive Guide to Artificial Intelligence. Addison-Wesley Professional.

Kumar, R., 2019. Machine Learning Quick Reference: Quick and Essential Machine Learning Hacks for Training Smart Data Models. Packt Publishing Ltd.

Lei, J., 2019. Cross-validation with confidence. J. Am. Stat. Assoc. 115 (532), 1–20.

Millstein, F., 2020. Convolutional Neural Networks in Python: Beginner's Guide to Convolutional Neural Networks in Python. Frank Millstein.

Moolayil, J., Moolayil, J., John, S., 2019. Learn Keras for Deep Neural Networks. Springer.

Müller, A.C., Guido, S., 2016. Introduction to Machine Learning With Python: A Guide for Data Scientists. O'Reilly Media, Inc.

Provost, F., Fawcett, T., 2013. Data Science for Business: What You Need to Know About Data Mining and Data-Analytic Thinking. O'Reilly Media, Inc.

Raschka, S., 2015. Python Machine Learning. Packt Publishing Ltd.

Raschka, S., Mirjalili, V., 2017. Python Machine Learning. Packt Publishing Ltd.

Richert, W., 2013. Building Machine Learning Systems With Python. Packt Publishing Ltd.

Sarkar, D., Bali, R., Sharma, T., 2018. Practical Machine Learning With Python. A Problem-Solvers Guide to Building Real-World Intelligent Systems. Apress, Berkely.

Shanmugamani, R., 2018. Deep Learning for Computer Vision: Expert Techniques to Train Advanced Neural Networks Using TensorFlow and Keras. Packt Publishing Ltd.

Swamynathan, M., 2019. Mastering Machine Learning With Python in Six Steps: A Practical Implementation Guide to Predictive Data Analytics Using Python. Apress.

Viswanathan, V., et al., 2016. R: Recipes for Analysis, Visualization and Machine Learning. Packt Publishing Ltd.

Zaman, H.B., et al., 2019. Advances in Visual Informatics: 6th International Visual Informatics Conference, IVIC 2019, Bangi, Malaysia, November 19–21, 2019, Proceedings. vol. 11870 Springer Nature.